Unit IV

Data Manipulations

By

Team – Essentials of Data Science

School of Computer Engineering,
MIT Academy of Engineering, Alandi(D.)

Significance of Pandas

- Pandas is a powerful and popular library in the field of data science. It provides easy-to-use data structures and data analysis tools for Python, making it an essential component of the data science ecosystem. Here are some key reasons for the significance of pandas in data science:

Significance of Pandas

- Data Manipulation: Pandas offers efficient data structures like DataFrames and Series, which allow for flexible and intuitive data manipulation. It provides a wide range of functions and methods for tasks such as filtering, selecting, transforming, and aggregating data. With pandas, data scientists can easily clean, preprocess, and reshape data to suit their analysis needs.
- Data Exploration and Analysis: Pandas simplifies the process of exploring and analyzing data. It provides various functions for descriptive statistics, data summarization, and data visualization. Pandas integrates well with other libraries like NumPy and Matplotlib, enabling comprehensive data analysis workflows.

Significance of Pandas

- Handling Missing Data: Real-world datasets often contain missing or incomplete data. Pandas provides effective tools for handling missing data, allowing users to fill in missing values or drop incomplete rows or columns. This feature is crucial for ensuring the quality and reliability of data analysis.
- Data Integration: Pandas facilitates the integration of data from different sources and formats. It supports reading and writing data from various file formats such as CSV, Excel, SQL databases, and more. This versatility makes it easy to import, export, and merge datasets, enabling data scientists to work with diverse data sets seamlessly.

Significance of Pandas

- Time Series Analysis: Pandas has robust support for time series data analysis. It offers specialized data structures like DateTimeIndex and functions for resampling, time shifting, and time-based operations. With pandas, data scientists can easily handle and analyze time-stamped data, which is commonly encountered in financial, economic, and sensor data analysis.
- Data Preparation for Machine Learning: In machine learning workflows, data preparation is a crucial step. Pandas simplifies this process by providing functions for feature selection, encoding categorical variables, scaling numerical features, and more. It helps data scientists prepare their datasets in a format suitable for training machine learning models.

Significance -- Data Loading

- Data loading is a crucial step in the data science workflow and holds significant importance in the field. Here are some key reasons why data loading is significant in data science:
- Accessing and Preparing Data
- Exploratory Data Analysis
- Feature Engineering
- Model Training and Evaluation
- Iterative Analysis and Model Improvement
- Reproducibility and Collaboration

Data Loading

- Pandas provides several methods for loading data from different sources. Here are some common ways to load data using pandas:
- CSV Files:
- import pandas as pd
- # Load a CSV file
- df = pd.read_csv('data.csv')

- # Load a CSV file with custom delimiter
- df = pd.read_csv('data.csv', delimiter=';')
- # Load a CSV file with specific columns
- df = pd.read_csv('data.csv', usecols=['col1', 'col2'])

Data Loading

- Excel File

- import pandas as pd
- # Load an Excel file
- df = pd.read_excel('data.xlsx')

- # Load a specific sheet from an Excel file
- df = pd.read_excel('data.xlsx', sheet_name='Sheet1')

- # Load specific columns from an Excel file
- df = pd.read_excel('data.xlsx', usecols=['col1', 'col2'])

Data Loading

- JSON File

- import pandas as pd

- # Load a JSON file
- df = pd.read_json('data.json')

- # Load specific JSON records from a file
- df = pd.read_json('data.json', lines=True)

Data Loading -- Example

CSV File

- Name,Age,Gender,Salary
- Alice,25,Female,50000
- Bob,30,Male,60000
- Charlie,35,Male,70000
- David,40,Male,80000
- Eve,45,Female,90000

Data Loading -- Example

Read CSV File

- import pandas as pd

- # Load the CSV file
- df = pd.read_csv('data.csv')

- # Display the DataFrame
- print(df)

Output

```
 Name  Age  Gender  Salary
0    Alice   25  Female   50000
1      Bob   30    Male   60000
2  Charlie   35    Male   70000
3    David   40    Male   80000
4      Eve   45  Female   90000
```

Significance -- Data Storage

- Data storage is a critical component in data science, playing a significant role in the entire data lifecycle. Here are some key reasons why data storage is significant in data science:
- Data Preservation
- Data Management
- Data Integration
- Scalability
- Data Security
- Data Sharing and Collaboration
- Reproducibility and Auditability
- Disaster Recovery and Business Continuity

Data Storage

- Pandas provides several methods for storing data to different formats. Here are some common ways to store data using pandas:

CSV File

- import pandas as pd

- # Save DataFrame to a CSV file
- df.to_csv('data.csv', index=False)

- # Save DataFrame to a CSV file with custom delimiter
- df.to_csv('data.csv', sep=';')

Data Storage

- Excel File

- import pandas as pd

- # Save DataFrame to an Excel file
- df.to_excel('data.xlsx', index=False)

- # Save DataFrame to an Excel file with specific sheet name
- df.to_excel('data.xlsx', sheet_name='Sheet1')

Data Storage

- JSON File

- import pandas as pd

- # Save DataFrame to a JSON file
- df.to_json('data.json', orient='records')

- # Save DataFrame to a JSON file with each record on a new line
- df.to_json('data.json', orient='records', lines=True)

Data Storage

- JSON File

- import pandas as pd

- # Save DataFrame to a JSON file
- df.to_json('data.json', orient='records')

- # Save DataFrame to a JSON file with each record on a new line
- df.to_json('data.json', orient='records', lines=True)

Data Storage --Example

```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'Age': [25, 30, 35, 40, 45],
    'Gender': ['Female', 'Male', 'Male', 'Male', 'Female'],
    'Salary': [50000, 60000, 70000, 80000, 90000]
}

df = pd.DataFrame(data)
```

Data Storage --Example

- df.to_csv('data.csv', index=False)

Output

```
Name,Age,Gender,Salary
Alice,25,Female,50000
Bob,30,Male,60000
Charlie,35,Male,70000
David,40,Male,80000
Eve,45,Female,90000
```

Significance -- Summarizing and Computing Descriptive Statistics

- Summarizing and computing descriptive statistics are essential tasks in data science that hold significant importance. Here are some key reasons why summarizing and computing descriptive statistics are significant in data science:
- Data Exploration and Understanding
- Data Cleaning and Preprocessing
- Data Visualization
- Data Comparison and Benchmarking
- Feature Selection and Dimensionality Reduction
- Model Input Preparation
- Data-driven Decision Making
- Communication and Reporting

Summarizing and Computing Descriptive Statistics

- Pandas provides a wide range of functions and methods for summarizing and computing descriptive statistics on data. Here are some commonly used techniques in pandas:

Summary Statistics

- import pandas as pd

- # Compute basic summary statistics
- df.describe()

- # Compute the mean of each column
- df.mean()

- # Compute the median of each column
- df.median()

- # Compute the maximum value of each column
- df.max()

- # Compute the minimum value of each column
- df.min()

Counting Values

- import pandas as pd

- # Count the occurrences of each unique value in a column
- df['column_name'].value_counts()

- # Count the total number of non-missing values in each column
- df.count()

Aggregation

- import pandas as pd

- # Compute the sum of values in each column
- df.sum()

- # Compute the maximum value in each column
- df.max()

- # Compute the minimum value in each column
- df.min()

- # Compute the average value in each column
- df.mean()

- # Compute the median value in each column
- df.median()

Group By Operations

- import pandas as pd

- # Group by a column and compute the sum for each group
- df.groupby('column_name').sum()

- # Group by multiple columns and compute the mean for each group
- df.groupby(['column1', 'column2']).mean()

- # Apply multiple aggregation functions to each group
- df.groupby('column_name').agg(['mean', 'max', 'min'])

Corelation and Covariance

- import pandas as pd

- # Compute the correlation between columns
- df.corr()

- # Compute the covariance between columns
- df.cov()

Quantiles

- import pandas as pd

- # Compute the quantiles of a column
- df['column_name'].quantile([0.25, 0.5, 0.75])

- # Compute the quantiles of multiple columns
- df[['column1', 'column2']].quantile([0.25, 0.5, 0.75])

Example

```
import pandas as pd

# Create a sample DataFrame
data = {'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'], 'Math': [90, 82, 95, 78, 88], 'Science': [85, 79, 92, 75, 90],   'English': [88, 85, 90, 80, 92]}
df = pd.DataFrame(data)

# Display the DataFrame
print(df)
```

Output:

```
 Name  Math  Science  English
0    Alice    90       85       88
1      Bob    82       79       85
2  Charlie    95       92       90
3    David    78       75       80
4      Eve    88       90       92
```

Example

- print(df.describe())

Output:

```
       Math    Science    English
count  5.000000  5.000000  5.000000
mean  86.600000 84.200000 87.000000
std    6.454972  6.719478  4.924429
min   78.000000 75.000000 80.000000
25%   82.000000 79.000000 85.000000
50%   88.000000 85.000000 88.000000
75%   90.000000 90.000000 90.000000
max   95.000000 92.000000 92.000000
```

Example

- print(df['Math'].value_counts())

Output:

```
88    1
78    1
82    1
90    1
95    1
Name: Math, dtype: int64
```

Example

- print(df.sum())
- print(df.mean())

Output:

```
Math      433
Science    421
English    435
dtype: int64
Math      86.6
Science    84.2
English    87.0
dtype: float64
```

Example

- print(df.groupby('Name').mean())

Output:

```
 Math  Science  English
Name
Alice     90      85      88
Bob       82      79      85
Charlie   95      92      90
David     78      75      80
Eve       88      90      92
```

Example

- print(df.corr())
- print(df.cov())

Output:

```
         Math   Science   English
Math     1.000000  0.950251  0.976981
Science  0.950251  1.000000  0.944911
English  0.976981  0.944911  1.000000

         Math    Science    English
Math     41.700   35.50000   43.50000
Science  35.500   45.70000   42.90000
English  43.500   42.90000   24.25000
```

Significance – Data Cleaning

- Data cleaning, also known as data cleansing or data scrubbing, is a critical step in the data science workflow. It involves identifying and correcting or removing errors, inconsistencies, and inaccuracies in the data. Here are some key reasons why data cleaning is significant in data science:
- Data Quality Assurance
- Accurate Analysis and Modeling
- Consistent Data Structure
- Handling Missing Data
- Outlier Detection and Treatment
- Data Security and Privacy
- Reproducibility and Collaboration
- Data Integration and Compatibility

Data Cleaning

- Data cleaning using pandas involves using the various functions and methods provided by the pandas library to identify and handle common data cleaning tasks. Here are some examples of data cleaning tasks and how they can be performed using pandas:

Handling Missing Values:

- To check for missing values in a DataFrame:

```
df.isnull()
```

To drop rows or columns with missing values:

```
df.dropna()  # Drops rows with any missing value
df.dropna(axis=1)  # Drops columns with any missing value
```

Data Cleaning

- Data cleaning using pandas involves using the various functions and methods provided by the pandas library to identify and handle common data cleaning tasks. Here are some examples of data cleaning tasks and how they can be performed using pandas:

Data Cleaning -- Handling Missing Values:

- To check for missing values in a DataFrame:

```
df.isnull()
```

- To drop rows or columns with missing values:

```
df.dropna()  # Drops rows with any missing value
df.dropna(axis=1)  # Drops columns with any missing value
```

To fill missing values with a specific value or strategy:

```
df.fillna(value)  # Fill missing values with a specific value
df.fillna(df.mean())  # Fill missing values with the column mean
```

Data Cleaning -- Handling Duplicate Values:

- To check for duplicate rows in a DataFrame:

df.duplicated()

- To drop duplicate rows:

df.drop_duplicates()

Data Cleaning -- Handling Outliers:

- To identify and remove outliers using z-score:

```
from scipy import stats
z_scores = stats.zscore(df['column'])
threshold = 3
df = df[(z_scores < threshold)]
```

Data Cleaning – Datatype Conversion

- To convert a column to a specific data type:

```
df['column'] = df['column'].astype('int')  # Convert to integer type
```

Data Cleaning – Handling Inconsistent Data:

- To replace values based on a condition:

```
df.loc[df['column'] == 'old_value', 'column'] = 'new_value'
```

- To standardize text data by converting to lowercase:

```
df['column'] = df['column'].str.lower()
```

Data Cleaning – Handling Text Cleaning:

- To remove leading or trailing whitespace from string columns:

```
df['column'] = df['column'].str.strip()
```

- To remove special characters or specific patterns from string columns using regular expressions:

```
import re

df['column'] = df['column'].str.replace('[^\w\s]', '')
```

Data Cleaning -- Example

```
import pandas as pd
# Create a sample DataFrame
data = {    'Name': ['John', 'Alice', 'Bob', 'Jane', 'Mike'],   'Age': [25, 28, None, 32, 40],   'Salary': [50000, 60000, 20000, 80000, 120000]}
df = pd.DataFrame(data)
# Check for missing values
print(df.isnull())
# Drop rows with missing values
df.dropna(inplace=True)
# Identify and remove outliers in the 'Salary' column using z-score
from scipy import stats
df = df[(np.abs(stats.zscore(df['Salary'])) < 3)]
# Convert 'Age' column to integer type
df['Age'] = df['Age'].astype(int)
 # Print the cleaned DataFrame
print(df)
```

Data Cleaning -- Example

- Output

- Name Age Salary
- 0 John 25 50000
- 1 Alice 28 60000
- 3 Jane 32 80000

Significance -- Data Preparation

- Data preparation is a crucial step in the data science workflow, and its significance cannot be overstated. It involves transforming raw data into a clean, organized, and suitable format that can be used for analysis and modeling. Here are some key reasons why data preparation is essential in data science:
- Data Quality
- Feature Engineering
- Data Integration
- Data Normalization and Scaling
- Handling Categorical Variables
- Data Reduction
- Outlier Detection and Treatment
- Reproducibility and Documentation

Data Preparation

1.Importing Data: Pandas allows you to read data from various file formats such as CSV, Excel, SQL databases, and more. You can use the **read_csv()**, **read_excel()**, or **read_sql()** functions to import data into a Pandas DataFrame.

2.Handling Missing Values: Missing values are a common issue in datasets. Pandas provides methods to handle missing data, such as **isna()**, **fillna()**, and **dropna()**. You can use these methods to identify missing values, fill them with appropriate values, or drop rows or columns with missing data.

```
# Check for missing values
df.isna()

# Fill missing values with a specific value
df.fillna(0)

# Drop rows with missing values
df.dropna()
```

Data Preparation

3.Data Cleaning: Pandas offers powerful functions for cleaning and transforming data. You can use methods like **replace()**, **strip()**, **lower()**, **upper()**, and regular expressions (**str.replace()**) to clean and standardize the data.

```
# Replace values
df.replace('old_value', 'new_value')

# Strip leading and trailing whitespace
df['column_name'].str.strip()

# Convert string to lowercase
df['column_name'].str.lower()

# Apply regular expression to replace values
df['column_name'] = df['column_name'].str.replace(r'\D',
'')
```

Data Preparation

4.Data Filtering and Selection: Pandas provides flexible methods to filter and select data based on specific conditions. You can use boolean indexing or the **query()** method to filter rows based on column values.

```
# Filter rows based on a condition
filtered_df = df[df['column_name'] > 10]

# Filter rows using query method
filtered_df = df.query('column_name > 10')

# Select specific columns
selected_columns = df[['column_name1', 'column_name2']]
```

Data Preparation

5.Data Transformation: Pandas allows you to transform data by adding new columns, applying mathematical operations, or grouping data. You can use the **assign()** method, mathematical operations (**+**, **-**, *****, **/**), or **groupby()** to perform transformations.

```
# Add a new column
df['new_column'] = df['column1'] + df['column2']

# Apply a mathematical operation to a column
df['column'] = df['column'] * 2

# Group data by a column and calculate aggregate
statistics
grouped_df = df.groupby('column_name').mean()
```

Data Preparation

6.Handling Categorical Data: Pandas provides methods to handle categorical variables. You can use **get_dummies()** for one-hot encoding or **astype()** to convert categorical variables to the appropriate data type.

```
# Perform one-hot encoding
encoded_df =
pd.get_dummies(df['categorical_column'])

# Convert a column to categorical type
df['categorical_column'] =
df['categorical_column'].astype('category')
```

Data Wrangling-joini

Data wrangling involves manipulating and reshaping data to transform it into a suitable format for analysis. Pandas
commo

1.Joining DataFrames:

- Merge: Combine two DataFrames based on common columns using **merge()**.
- Concatenate: Append or stack DataFrames vertically or horizontally using **concat()**.

import pandas as pd

```
import pandas as pd

df1 = pd.DataFrame({'key': ['A', 'B', 'C', 'D'],
                    'value1': [1, 2, 3, 4]})

df2 = pd.DataFrame({'key': ['B', 'D', 'E', 'F'],
                    'value2': [5, 6, 7, 8]})

merged_df = pd.merge(df1, df2, on='key')

print(merged_df)
```

```
  key value1 value2
0 B 2 5
1 D 4 6
```

Data Wrangling-combine

Combining DataFrames:

- Merge: Join multiple DataFrames based on common columns using **merge()**.
- Concatenate: Combine multiple DataFrames vertically or horizontally using **concat()**.

```
import pandas as pd

# Create two DataFrames
df1 = pd.DataFrame({'ID': [1, 2, 3],
                    'Name': ['Alice', 'Bob', 'Charlie']})

df2 = pd.DataFrame({'ID': [4, 5, 6],
                    'Name': ['Dave', 'Eve', 'Frank']})

# Concatenate the DataFrames vertically
concatenated_df = pd.concat([df1, df2], axis=0)

print(concatenated_df)
```

```
  ID Name
0 1 Alice 1
2 Bob
2 3 Charlie
0 4 Dave
1 5 Eve
2 6 Frank
```

Data Wrangling-Reshaping

Reshaping Data:

- Pivot: Reshape data from long to wide format using **pivot()**.
- Melt: Unpivot data from wide to long format using **melt()**.
- Stack/Unstack: Reshape data between hierarchical and tabular formats using **stack()** and **unstack()**.

```
import pandas as pd
```

```
# Reshaping using pivot
df = pd.DataFrame({'City': ['Tokyo', 'Paris', 'Tokyo', 'Paris'],
                   'Year': [2020, 2020, 2021, 2021],
                   'Temperature': [25, 20, 28, 22]})
pivot_df = df.pivot(index='Year', columns='City',
values='Temperature')

print(pivot_df)
```

```
City Paris Tokyo
Year 2020 20 25
     2021 22 28
```

Data Wrangling-Reshaping

```
import pandas as pd

df = pd.DataFrame({'ID': [1, 2, 3],
                   'Math': [80, 90, 85],
                   'Science': [70, 80, 75]})

melted_df = df.melt(id_vars='ID', var_name='Subject',
value_name='Score')
```

ID Subject Score 0 1
Math 80
1 2 Math 90
2 3 Math 85
3 1 Science 70
4 2 Science 80
5 3 Science 75

Data Wrangling

```
import pandas as pd

# Create a DataFrame
data = {
    'ID': [1, 2, 3],
    'Math': [80, 90, 85],
    'Science': [70, 80, 75],
}
df = pd.DataFrame(data)

# Perform stack operation
stacked_df = df.set_index('ID').stack()

# Perform unstack operation
unstacked_df = stacked_df.unstack()
```

Output of stack

```
ID
1   Math       80
    Science    70
2   Math       90
    Science    80
3   Math       85
    Science    75
dtype: int64
```

Output of unstack

```
Math  Science
ID
1    80       70
2    90       80
3    85       75
```

Data Transformation

In data science, data transformation refers to the process of converting or modifying the original data to make it more suitable for analysis or modeling. Data transformation involves applying various techniques to modify the structure, format, or distribution of the data. The goal of data transformation is to improve data quality, handle outliers or missing values, normalize variables, capture non-linear relationships, and meet the assumptions of statistical methods.

Scaling and Normalization:

Encoding Categorical Variables:

Logarithmic or Power Transformations:

Binning and Discretization:

Data Transformation

```
import pandas as pd

# Create a DataFrame
data = {
    'ID': [1, 2, 3, 4, 5],
    'Category': ['A', 'B', 'A', 'C', 'B'],
    'Value': [10, 15, 20, 25, 30]
}
df = pd.DataFrame(data)
print(df)
# Applying transformation
df['Value_squared'] = df['Value'] ** 2
df['Category_upper'] =
df['Category'].str.upper()
df['Category_numeric'] =
df['Category'].map({'A': 1, 'B': 2, 'C': 3})
print(df)
```

```
   ID Category  Value
0   1        A     10
1   2        B     15
2   3        A     20
3   4        C     25
4   5        B     30
   ID Category  Value  Value_squared Category_upper  Category_numeric
0   1        A     10            100             A                 1
1   2        B     15            225             B                 2
2   3        A     20            400             A                 1
3   4        C     25            625             C                 3
4   5        B     30            900             B                 2
```

Data Aggregation

Data aggregation in data science refers to the process of combining and summarizing data to obtain meaningful insights or derive useful information. Aggregation involves grouping data based on certain criteria and applying aggregation functions to calculate summary statistics or perform calculations on the grouped data. Aggregation helps in condensing large datasets into more manageable and understandable forms for analysis and reporting.

Here are some key aspects of data aggregation in data science:

1.Grouping Data: Data aggregation starts with grouping the data based on one or more variables. The data is divided into subsets based on the values of these variables, creating groups or categories for analysis.

2.Aggregation Functions: Aggregation functions are applied to the grouped data to calculate summary statistics or perform calculations on each group. Common aggregation functions include sum, mean, median, count, min, max, standard deviation, and variance.

Data Aggregation

3. Grouped Operations: Aggregation allows performing operations specific to each group. These operations can involve calculating group-specific metrics, applying custom functions, or performing calculations using multiple variables within each group.

4. Hierarchical Aggregation: Aggregation can be performed at different levels of hierarchy. For example, data can be aggregated at the overall dataset level, as well as within subsets defined by multiple variables or combinations of variables.

5. Aggregation on Time Series Data: Time-based aggregation is commonly used in analyzing time series data. It involves grouping data into intervals such as days, weeks, months, or years and calculating aggregate metrics or summary statistics within each interval.

6. Pivot Tables: Pivot tables are a powerful tool for data aggregation. They allow summarizing data by grouping variables and displaying the results in a tabular format, where rows represent one variable, columns represent another variable, and the values are aggregated based on specified functions.

7.Data Visualization: Aggregated data is often visualized using charts, graphs, or other visual representations. Visualizing aggregated data helps in understanding patterns, trends, or comparisons between different groups.

Data Aggregation

```
import pandas as pd

# Create a DataFrame
data = {
    'Category': ['A', 'A', 'B', 'B', 'A', 'B'],
    'Value': [10, 15, 20, 25, 30, 35]
}
df = pd.DataFrame(data)

# Perform aggregation
grouped_df = df.groupby('Category').agg({'Value': ['sum', 'mean', 'count']})
```

	Value		
	sum	mean	count
Category			
A	55	18.333333	3
B	80	26.666667	3

Group Operations

Group operations in pandas involve performing calculations or transformations on grouped data. Pandas provides several methods and functions to facilitate group operations. Here are some commonly used group operations in pandas:

1.Aggregation: Aggregation involves calculating summary statistics on grouped data. Common aggregation functions include **sum()**, **mean()**, **median()**, **count()**, **min()**, **max()**, **std()**, and **var()**. These functions can be applied to specific columns or the entire DataFrame.

2.Transformation: Transformation involves performing calculations on groups and returning data aligned with the original DataFrame. The **transform()** method is commonly used for this purpose.

3.Filtering: Filtering allows you to select specific groups based on certain conditions. The **filter()** method is used to apply a filtering condition to each group and return only the groups that meet the condition. This can be helpful for removing groups that don't satisfy specific criteria or for selecting groups with a minimum number of observations.

Group Operations

4.Applying Custom Functions: Pandas provides the **apply()** method to apply custom functions to grouped data. This allows you to perform complex operations or calculations on each group. You can define your own function and use it with **apply()** to process each group independently.

5.Iterating over Groups: You can iterate over groups using the **groupby()** function. This allows you to access each group individually and perform operations or calculations on them. However, iterating over groups should be avoided whenever possible, as it is often slower compared to vectorized operations.

6.Pivot Tables: Pivot tables in pandas provide a way to summarize and aggregate data in a tabular format. The **pivot_table()** function allows you to specify the index, columns, and values to be aggregated. Pivot tables are useful for analyzing multidimensional data and can be customized to display the desired summary statistics.

Group Operations

```
import pandas as pd

# Create a DataFrame
data = {
    'Category': ['A', 'A', 'B', 'B', 'A', 'B'],
    'Value': [10, 15, 20, 25, 30, 35],
    'Flag': [True, False, True, True, False, True]
}
df = pd.DataFrame(data)

# Grouping by 'Category'
grouped = df.groupby('Category')

# Aggregation: calculating sum, mean, and count
aggregated = grouped.agg({'Value': ['sum', 'mean', 'count']})
print("Aggregation:")
print(aggregated)
```

Group Operations

```
# Transformation: subtracting group mean from 'Value'
transformed = grouped['Value'].transform(lambda x: x - x.mean())
df['Transformed'] = transformed
print("\nTransformation:")
print(df)

# Filtering: selecting groups with a count greater than 1
filtered = grouped.filter(lambda x: len(x) > 1)
print("\nFiltering:")
print(filtered)
```

Group Operations

```
# Applying custom function: calculating the difference between max and min values for each group
def custom_function(x):
    return x.max() - x.min()

custom_applied = grouped['Value'].apply(custom_function)
print("\nCustom Function:")
print(custom_applied)

# Iterating over groups: calculating the mean of each group
print("\nIterating over groups:")
for group, data in grouped:
    group_mean = data['Value'].mean()
    print(f"Group {group}: Mean = {group_mean}")

# Pivot table: calculating the mean 'Value' for each category and flag combination
pivot_table = pd.pivot_table(df, values='Value', index='Category', columns='Flag', aggfunc='mean')
print("\nPivot Table:")
print(pivot_table)
```

Group Operations

```
Aggregation:
         Value
           sum       mean count
Category
A           55  18.333333     3
B           80  26.666667     3

Transformation:
  Category  Value   Flag  Transformed
0        A     10   True    -8.333333
1        A     15  False    -3.333333
2        B     20   True    -6.666667
3        B     25   True    -1.666667
4        A     30  False    11.666667
5        B     35   True     8.333333

Filtering:
  Category  Value   Flag  Transformed
0        A     10   True    -8.333333
1        A     15  False    -3.333333
2        B     20   True    -6.666667
3        B     25   True    -1.666667
4        A     30  False    11.666667
5        B     35   True     8.333333
```

```
Custom Function:
Category
A    20
B    15
Name: Value, dtype: int64

Iterating over groups:
Group A: Mean = 18.333333333333332
Group B: Mean = 26.666666666666668

Pivot Table:
Flag      False      True
Category
A          22.5  10.000000
B           NaN  26.666667
```

iloc and loc

1.loc:

- **loc** is primarily label-based indexing, which means you can select data based on the labels of rows or columns.
- It accepts a label or a boolean array for row indexing and a label or a list of labels for column indexing.
- The syntax for using **loc** is **df.loc[row_indexer, column_indexer]**.
- The row and column indexers can be single labels, lists, slices, or boolean arrays.

2.iloc:

- **iloc** is primarily integer-based indexing, which means you can select data based on the integer positions of rows or columns.
- It accepts integer values or boolean arrays for row and column indexing.
- The syntax for using **iloc** is **df.iloc[row_indexer, column_indexer]**.
- The row and column indexers can be single integers, lists of integers, slices, or boolean arrays.

iloc and loc

1.loc:

- **loc** is primarily label-based indexing, which means you can select data based on the labels of rows or columns.
- It accepts a label or a boolean array for row indexing and a label or a list of labels for column indexing.
- The syntax for using **loc** is **df.loc[row_indexer, column_indexer]**.
- The row and column indexers can be single labels, lists, slices, or boolean arrays.

2.iloc:

- **iloc** is primarily integer-based indexing, which means you can select data based on the integer positions of rows or columns.
- It accepts integer values or boolean arrays for row and column indexing.
- The syntax for using **iloc** is **df.iloc[row_indexer, column_indexer]**.
- The row and column indexers can be single integers, lists of integers, slices, or boolean arrays.

iloc and loc

```
import pandas as pd
data = {
    'Name': ['John', 'Alice', 'Bob', 'Emma', 'Tom'],
    'Age': [25, 30, 35, 28, 32],
    'City': ['New York', 'London', 'Paris', 'Sydney', 'Tokyo']
}
df = pd.DataFrame(data)

# Using loc to select data Select rows with index labels 1 and 3, and columns 'Name' and 'City'
selected_loc = df.loc[[1, 3], ['Name', 'City']]
print("Using loc:")
print(selected_loc)

# Using iloc to select data Select rows at integer positions 1 and 3, and columns at integer
positions 0 and 2
selected_iloc = df.iloc[[1, 3], [0, 2]]
print("\nUsing iloc:")
print(selected_iloc)
```

iloc and loc

```
# importing the module
import pandas as pd


# creating a sample dataframe
data = pd.DataFrame({'Brand': ['Maruti', 'Hyundai', 'Tata',
                               'Mahindra', 'Maruti', 'Hyundai',
                               'Renault', 'Tata', 'Maruti'],
                     'Year': [2012, 2014, 2011, 2015, 2012,
                              2016, 2014, 2018, 2019],
                     'Kms Driven': [50000, 30000, 60000,
                                    25000, 10000, 46000,
                                    31000, 15000, 12000],
                     'City': ['Gurgaon', 'Delhi', 'Mumbai',
                              'Delhi', 'Mumbai', 'Delhi',
                              'Mumbai', 'Chennai',  'Ghaziabad'],
                     'Mileage':  [28, 27, 25, 26, 28,
                                  29, 24, 21, 24]})


# displaying the DataFrame
display(data)
```

iloc and loc

```
# selecting cars with brand 'Maruti' and Mileage > 25
display(data.loc[(data.Brand == 'Maruti') & (data.Mileage > 25)])

# selecting range of rows from 2 to 5
display(data.loc[2: 5])

# updating values of Mileage if Year < 2015
data.loc[(data.Year < 2015), ['Mileage']] = 22
display(data)

# selecting 0th, 2nd, 4th, and 7th index rows
display(data.iloc[[0, 2, 4, 7]])

# selecting rows from 1 to 4 and columns from 2 to 4
display(data.iloc[1: 5, 2: 5])
```